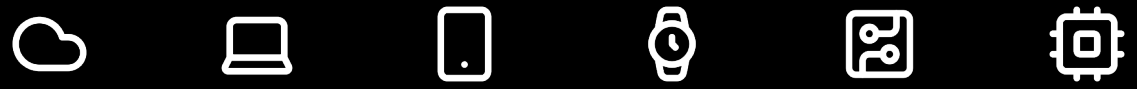


Mobile and Embedded Computing

Lecture 12: gRPC, platform channels & FFI, Kotlin Multiplatform

Dinu-Ștefan Rusu

ORIENTATION

Who's teaching this course

Dinu-Ștefan Rusu

dinu_stefan.rusu@upb.ro
Microsoft Teams course channel
rusudinu.com

45+

Flutter apps shipped

400k

installs across them

Appeared in, spoken at & partnered with

TODAY

What you should leave with

Cross the network	A .proto contract, generated stubs, a Go server and a Dart client, and the transport under them
Cross into native	MethodChannel and dart:ffi: what each one really costs, measured on your own device
Share code across platforms	A Kotlin Multiplatform module with expect/actual, consumed by Android and iOS
Place the boundary	Run time or compile time, and what each choice costs your team

Four boundaries, one lecture

Between machines

Your app and a server, one round trip apart. gRPC, protobuf, HTTP/2 and HTTP/3.

Between languages

Dart asking Kotlin or Swift for something. Platform channels: messages, encoded and posted.

Between runtimes

Dart calling C in the same process. dart:ffi: no encoding, no hop, no safety net.

Between platforms

One body of logic, two operating systems. Kotlin Multiplatform, compiled natively for each.

Every boundary has a cost: serialization, a thread hop, a round trip, or a second implementation. The engineering is choosing which one you pay.

BOUNDARY ONE

Between machines

gRPC, Protocol Buffers, and the transport underneath them

RPC and REST answer different questions

REST models nouns.

You name resources and use a small fixed set of verbs on them. It is a wonderful fit for a public API that strangers must discover from a URL alone.

RPC models verbs.

You name the operation your app actually needs, such as GetDashboard or SubmitReading, and call it as if it were a local function. The client and the server agree on a list of procedures, not a URL space.

On a phone the difference is round trips.

A resource-shaped API often needs three or four sequential requests to fill one screen, and on cellular each one costs 40–200 ms before any byte of your data moves.

```
# REST: four requests, four latencies
GET /users/42
GET /users/42/devices
GET /devices/9/readings?limit=20
GET /devices/9/alerts

# RPC: one call, one round trip,
# one response shaped like the screen
GetDashboard(user_id: 42) -> Dashboard
```

Neither one is “modern”. gRPC wins where you own both ends; REST wins where you do not.

Design the call around the screen, not around your database tables. The round trip is the expensive part.

Contract first: the .proto file

```
syntax = "proto3";
package telemetry.v1;
option go_package = "example.com/telemetry/gen/telemetryv1";

message Reading {
  string device_id        = 1;   // numbers 1..15 cost one tag byte,
  double temp_c           = 2;   // spend them on your hottest fields
  int64  taken_at_unix_ms = 3;
  bool   from_cache       = 4;
}

message SubmitReadingRequest  { Reading reading = 1; }
message SubmitReadingResponse { string stored_id = 1; }
message WatchDeviceRequest    { string device_id = 1; }

service Telemetry {
  rpc SubmitReading(SubmitReadingRequest) returns (SubmitReadingResponse);
  rpc WatchDevice(WatchDeviceRequest) returns (stream Reading);
}
```

One file, three teams.

The .proto is the only source of truth. Backend, Android and iOS generate from it and build in parallel.

The numbers are the wire format.

Field names never travel. The tag number does, so it can never change.

Enums need a zero.

In proto3 an absent enum decodes as 0, so 0 must mean UNSPECIFIED.

Version the package,

not the file: telemetry.v1 lets you ship a v2 alongside it.

Generating the stubs with protoc

```
# --- Go: two plugins, messages and service ---
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest

protoc -Iproto \
  --go_out=gen      --go_opt=paths=source_relative \
  --go-grpc_out=gen --go-grpc_opt=paths=source_relative \
  proto/telemetry/v1/telemetry.proto

# --- Dart: the plugin is a Dart global executable ---
dart pub global activate protoc_plugin
export PATH="$PATH:$HOME/.pub-cache/bin"

protoc -Iproto --dart_out=grpc:lib/gen \
  proto/telemetry/v1/telemetry.proto
```

What comes out

Reading, SubmitReadingRequest… as real classes with typed getters, equality and a binary codec.

TelemetryClient: the client stub you call.

TelemetryServiceBase / UnimplementedTelemetryServer: the skeleton you implement.

Generated code is build output.

Never edit it. Either commit it and regenerate on change, or run protoc in CI. Pick one and write it down.

pubspec.yaml needs protobuf and grpc.

The compiler writes the parsing code, which is the code you would otherwise get subtly wrong.

A Go server for that service

Unary: the generated interface, implemented

```
type server struct {
    pb.UnimplementedTelemetryServer   // new RPCs keep compiling
    store *Store
}
func (s *server) SubmitReading(ctx context.Context,
    req *pb.SubmitReadingRequest) (*pb.SubmitReadingResponse, error) {
    r := req.GetReading()
    if r.GetDeviceId() == "" {
        return nil, status.Error(codes.InvalidArgument, "device_id")
    }
    id, err := s.store.Save(ctx, r)   // ctx = the client's deadline
    if err != nil {
        return nil, status.Error(codes.Internal, "save failed")
    }
    return &pb.SubmitReadingResponse{StoredId: id}, nil
}
func main() {
    lis, _ := net.Listen("tcp", ":50051")
    g := grpc.NewServer()
    pb.RegisterTelemetryServer(g, &server{store: NewStore()})
    log.Fatal(g.Serve(lis))
}
```

Server streaming

```
func (s *server) WatchDevice(
    req *pb.WatchDeviceRequest,
    stream pb.Telemetry_WatchDeviceServer,
) error {
    ctx := stream.Context()
    for r := range s.store.Sub(ctx, req.DeviceId) {
        if err := stream.Send(r); err != nil {
            return err
        }
    }
    return nil   // ends the stream cleanly
}
```

The same Go you met in Lecture 5,

now speaking a generated interface instead of hand-written JSON handlers.

ctx is the deadline.

When the phone gives up after 2 s, ctx is canceled here and the query is abandoned, so no work continues for a call nobody is waiting on.

Errors are codes, not strings:

InvalidArgument, NotFound, Unauthenticated, Unavailable. The client branches on them.

The generated Dart client

Unary: a deadline and an auth header

```
final channel = ClientChannel('api.example.com', port: 443,
  options: const ChannelOptions(
    credentials: ChannelCredentials.secure(),
    idleTimeout: Duration(seconds: 30),  // let an idle socket close
  ));
final client = TelemetryClient(channel);

try {
  final res = await client.submitReading(
    SubmitReadingRequest(reading: Reading()
      ..deviceId = id
      ..tempC = 21.5
      ..takenAtUnixMs = Int64(nowMs)),
    options: CallOptions(
      timeout: const Duration(seconds: 2),
      metadata: {'authorization': 'Bearer $token'},
    ),
  );
  debugPrint(res.storedId);
} on GrpcError catch (e) {
  if (e.code == StatusCode.deadlineExceeded) { /* retry */ }
}
```

Server streaming is just a Dart Stream

```
final sub = client
    .watchDevice(WatchDeviceRequest()..deviceId = id)
    .listen(_onReading, onError: _onError);

@override
void dispose() {
  sub.cancel();          // stop the server sending
  channel.shutdown();    // and let the radio sleep
  super.dispose();
}
```

It looks like a local call.

That is the point of RPC, and the danger. It is still the network.

Always set a deadline.

Without one, a stalled call holds a UI state forever.

Cancel in dispose().

An open stream keeps the radio and the server busy for a screen the user already left.

Why protobuf is smaller, and by how much

Four mechanisms, all of them structural:

It is binary: no quotes, commas, braces or whitespace, and numbers are not re-rendered as text.

Fields are identified by number, not name: a tag byte packs the field number and a 3-bit wire type, so "taken_at_unix_ms" costs 1 byte on the wire instead of 18.

Integers are varints: 7 bits of value per byte, so 300 is AC 02 rather than four fixed bytes, and small numbers cost one byte.

Nothing is sent for a field left at its default.

How much smaller? That depends entirely on your data.

A message of short integers with long field names shrinks enormously. A message that is one long UTF-8 string barely shrinks at all, because the string is the same bytes in both formats. And gzip, which almost every JSON API and gRPC channel already applies, compresses away exactly the repetition protobuf never sends, narrowing the gap further.

```
# JSON: 38 bytes, key names ride along
{"device_id":"esp32-a1","temp_c":21.5}

# protobuf: 19 bytes for this message
0A 08 65 73 70 33 32 2D 61 31
#  |  |  \__ 'esp32-a1' ______/
#  |  len=8
#  field 1, wire type 2 (LEN)
11 00 00 00 00 00 80 35 40
#  field 2, wire type 1 (I64) = 21.5

# varint: 300 -> AC 02   (2 bytes, not 4)
```

Quote the mechanism, not a multiplier. “3–10× smaller” and “30–50% smaller” are both somebody else's payload. Measure yours.

Schema evolution without breaking old apps

```
message Reading {
  string device_id        = 1;
  double temp_c           = 2;
  int64  taken_at_unix_ms = 3;
  bool   from_cache       = 4;

  double humidity         = 5;   // added in v2
  optional int32 rssi     = 6;   // explicit presence:
                                 // tells 0 apart from absent

  reserved 7;                    // 7 was battery_pct, deleted.
  reserved "battery_pct";        // never reuse a number or a name
}
```

Users do not update. A year-old build is still on the network, and the wire format is the compatibility contract.

Old app, new server (forward).

The parser hits tag 5, reads its wire type, and knows exactly how many bytes to skip. It does not crash; proto3 even keeps the unknown bytes and re-emits them if it echoes the message back.

New server, old app (backward).

The missing field decodes to its default: 0, empty string, empty list. Which means “absent” and “zero” are the same thing unless you mark the field optional.

The rules that keep both true:

never renumber a field, never reuse a deleted number, never change a field's type, and reserve what you remove.

Four call shapes, not one

Unary

```
rpc GetDevice(GetDeviceRequest) returns (Device);
```

One request, one response. The default, and what almost every screen load should be.

Server streaming

```
rpc WatchDevice(WatchDeviceRequest) returns (stream Reading);
```

One request, many responses. Live readings, a feed, or progress on a long job, instead of polling.

Client streaming

```
rpc UploadBatch(stream Reading) returns (UploadSummary);
```

Many requests, one response. Flush a queue of offline readings, upload a file in chunks, and get one acknowledgement.

Bidirectional streaming

```
rpc Session(stream ClientMsg) returns (stream ServerMsg);
```

Both directions, independently, on one connection. Chat, presence, live location: the WebSocket case from Lecture 8, with a schema.

The shape is part of the contract. Changing a method from unary to streaming is a breaking change: it changes the generated signature on every client.

TRANSPORT

HTTP/2: one connection, many streams

HTTP/1.1 gives you one request at a time per connection. Browsers hid that by opening six connections; a phone pays for every one of them in handshakes and radio time.

HTTP/2 splits everything into binary frames tagged with a stream id, so many calls interleave over a single TCP connection. gRPC does not add this. gRPC is defined on top of it: one RPC is one HTTP/2 stream.

HPACK removes the other tax. Headers go into a static table of common names and a dynamic table shared by both ends, so a repeated Authorization header costs an index, not 800 bytes.

What HTTP/2 does not fix: TCP still delivers in order.

One lost packet stalls every stream on that connection until it is retransmitted. That is head-of-line blocking moved down a layer, and a lossy mobile link is where it shows.

One TCP + TLS connection
handshake once, reuse for every call

Binary framing
HEADERS and DATA frames, stream-tagged

Many streams in flight
interleaved, independently flow-controlled

HPACK
repeated headers cost a table index

One connection, kept warm. That is most of gRPC's mobile advantage, before a single byte of protobuf is counted.

TRANSPORT

HTTP/3 and QUIC: the mobile case

QUIC moves the whole thing onto UDP and implements streams itself. A lost packet now stalls only the stream it belonged to, so the head-of-line blocking that HTTP/2 pushed into TCP goes away.

TLS 1.3 is folded into the transport handshake: a new connection is one round trip, and a resumed one can be zero, with the first request riding along with the handshake. On a 150 ms link that is a visible difference.

The feature that only matters on a phone: connection migration.

A QUIC connection is identified by a connection ID, not by the IP/port pair. Walk out of Wi-Fi onto 5G and the connection survives; a TCP connection dies and every stream on it restarts.

Where gRPC stands today:

the wire protocol is specified over HTTP/2. HTTP/3 usually reaches you at the edge, where a load balancer or CDN speaks HTTP/3 to the device and HTTP/2 to your service, or through Cronet on Android. Treat it as a deployment choice, not a code change.

QUIC over UDP
independent streams, no shared stall

1-RTT handshake
0-RTT when resuming a session

Connection IDs
survives Wi-Fi → 5G handover

In practice
HTTP/3 at the edge, HTTP/2 behind it

Handshakes and reconnects are the mobile cost. QUIC addresses both, which is why it belongs in the same argument as the radio tail.

Deadlines, cancellation and the radio tail

A deadline is not a client-side timeout.

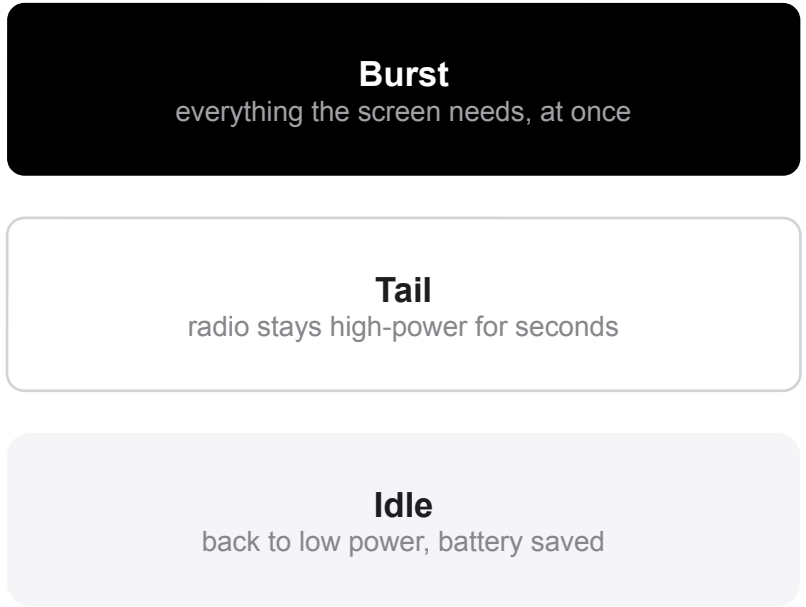
It travels with the call and propagates through every service behind it. When it expires, work stops everywhere, so nothing keeps burning CPU for a screen nobody is looking at.

The radio does not switch off immediately.

After a transmission the cellular radio stays in a high-power state for several seconds before stepping down. That period is the tail, and its duration is set by the network operator, not by you. Four chatty requests spread over ten seconds keep it awake the whole time; one multiplexed burst lets it sleep.

Keepalive pings are not free.

They hold the connection open and wake the radio. Tune the interval; do not leave a 10-second ping running in the background.



iOS caveat. A suspended app gets no CPU and its sockets are torn down, so a long-lived gRPC stream does not survive backgrounding. URLSession background transfers cover plain HTTP uploads and downloads only, and cannot carry a gRPC stream. Design for it: reconnect on resume, use a silent push or BGTaskScheduler to be woken, and make the server's state resumable. Android's Doze and App Standby do the same thing with different names.

Batch, set a deadline, cancel on dispose. Three habits that show up as battery life in a review.

gRPC-Web and Connect: the browser problem

A browser cannot speak gRPC.

Not for lack of trying: fetch and XHR give JavaScript no way to control HTTP/2 frames, and no way to read trailers. gRPC delivers the call's status in a trailer, after the body. Full-duplex streaming is not available either.

gRPC-Web

is a different wire encoding that puts the trailers in the body, works over HTTP/1.1, and supports unary and server-streaming only. It needs a translating proxy in front of your service, such as Envoy or grpcwebproxy.

Connect

is the pragmatic successor: one Go handler serves the gRPC, gRPC-Web and Connect protocols on the same route, and the Connect protocol is plain HTTP that curl can call. No proxy in the path.

Why you care in this course:

your Flutter app builds for web too, and that build cannot open a raw gRPC connection.

```
// one Go server, three protocols
mux := http.NewServeMux()
path, h := telemetryv1connect.
    NewTelemetryHandler(&server{})
mux.Handle(path, h)

// h2c: HTTP/2 without TLS, for local dev
http.ListenAndServe(":8080",
    h2c.NewHandler(mux, &http2.Server{}))
```

Mobile keeps the fast path; the web build gets a protocol it can actually speak.

The boundary has a shape here too: what the browser sandbox will let you send is part of your architecture.

BOUNDARIES TWO AND THREE

Between languages and runtimes

Platform channels, dart:ffi, and what each one costs

INSIDE THE APP

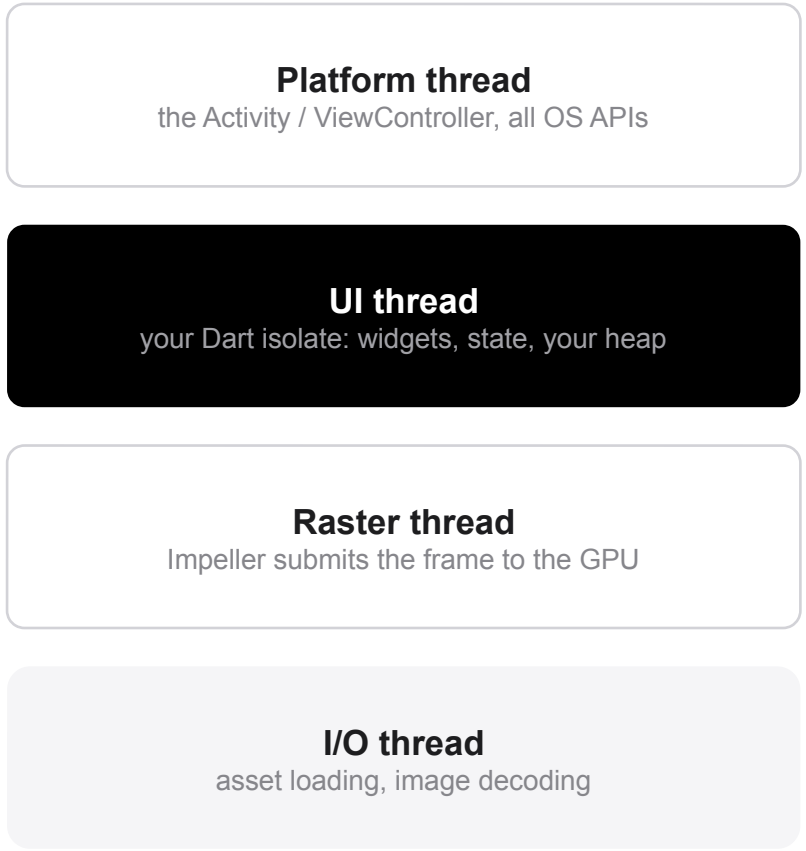
Two runtimes sharing one process

Your Flutter app is a native application. On Android an Activity, on iOS a UIViewController. That embedder loads the Flutter engine and hands it a surface, a message loop and system events.

Inside that process there are two managed worlds. Your Dart code runs in an isolate with its own heap and its own garbage collector. The platform runtime, ART on Android and the Objective-C/Swift runtime on iOS, has its own heap and its own rules. Neither can see the other's objects.

So every crossing is one of exactly two things:

encode a message and hand it to the other thread (a platform channel), or drop to the one thing both sides already agree on, the C ABI (dart:ffi).



Different heaps, different threads. Everything in this section is a consequence of those two facts.

MethodChannel, both sides

Dart

```
class Battery {
  static const _ch = MethodChannel('dev.upb.mec/battery');

  static Future<int> level() async {
    try {
      return await _ch.invokeMethod<int>('getLevel') ?? -1;
    } on PlatformException catch (e) {
      // e.code is the string the native side chose
      debugPrint('battery failed: ${e.code}');
      return -1;
    } on MissingPluginException {
      return -1;   // not registered on this platform
    }
  }
}
```

The channel name is a string on both sides, so a typo is a runtime MissingPluginException, not a compile error.

The handler runs on the platform's main thread: do the work elsewhere and call result later. EventChannel for streams.

Kotlin: MainActivity

```
override fun configureFlutterEngine(engine: FlutterEngine) {
  super.configureFlutterEngine(engine)
  val ch = MethodChannel(engine.dartExecutor.binaryMessenger,
                         "dev.upb.mec/battery")
  ch.setMethodCallHandler { call, result ->
    if (call.method == "getLevel") result.success(level())
    else result.notImplemented()
  }
}
```

Swift: AppDelegate

```
let ch = FlutterMethodChannel(name: "dev.upb.mec/battery",
        binaryMessenger: controller.binaryMessenger)
ch.setMethodCallHandler { call, result in
  guard call.method == "getLevel" else {
    result(FlutterMethodNotImplemented); return
  }
  result(Int(UIDevice.current.batteryLevel * 100))
}
```

What a channel call actually costs

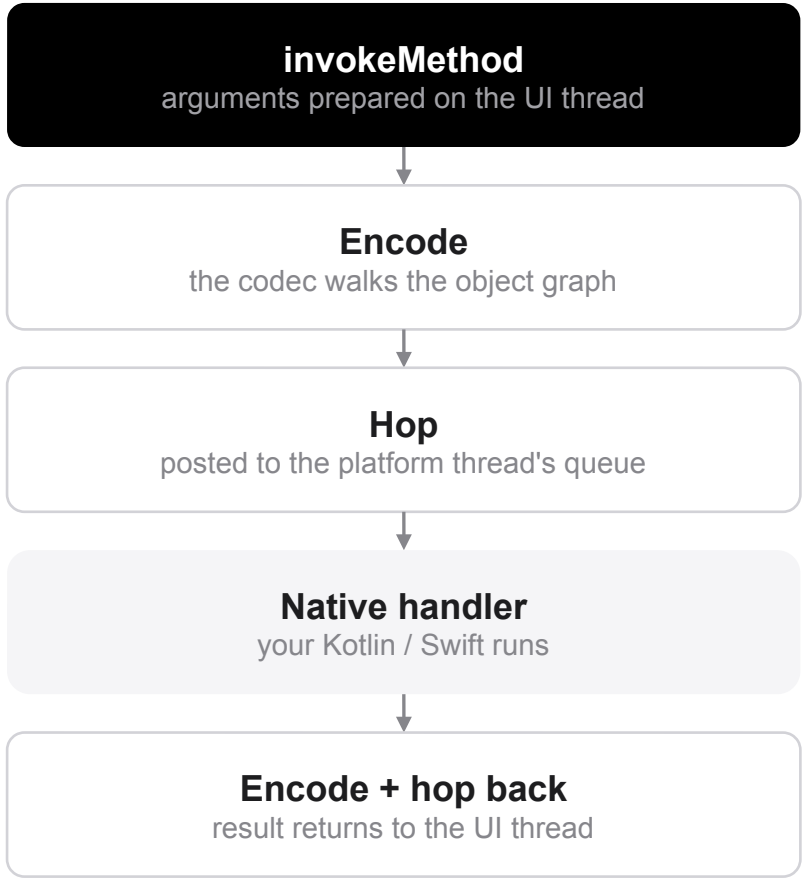
Two things are being paid for, and only one of them is constant.

The thread hops are a roughly fixed overhead: two queue posts and two context switches, whose cost depends on how busy those threads already are. Because the call is asynchronous it also inherits the UI thread's queue: if a frame is being built, your result waits behind it. That is where tail latency comes from.

The encoding is not fixed at all. StandardMessageCodec recurses over the object graph and writes every element, so the cost grows linearly with what you send. A map of five strings costs nothing; a list of 100,000 doubles is a different matter.

The Android trap:

a Dart List<double> decodes into 100,000 boxed java.lang.Double objects, and the garbage collector notices. Send a Float64List or Uint8List instead: the codec has a fast path that passes typed data through as a raw buffer, unboxed and copied once.



A channel call is a message, not a function call. Its cost scales with what you put in the message.

Measuring channel and FFI call cost

```
// release build, on a real device, never the simulator
const ch = MethodChannel('dev.upb.mec/bench');
const n = 10000;
// 1. warm up: the first call also sets the channel up
for (var i = 0; i < 200; i++) { await ch.invokeMethod('noop'); }

// 2. empty round trip: the floor for a channel call
var sw = Stopwatch()..start();
for (var i = 0; i < n; i++) { await ch.invokeMethod('noop'); }
print('channel noop: ${sw.elapsedMicroseconds / n} us/call');

// 3. the slope: repeat with 1k, 16k, then 256k doubles
final payload = Float64List(1024);
sw = Stopwatch()..start();
for (var i = 0; i < n; i++) { await ch.invokeMethod('echo', payload); }
print('channel 1k:   ${sw.elapsedMicroseconds / n} us/call');

// 4. the same nothing, through FFI: a direct C call
final noop = lib.lookupFunction<Void Function(), void Function()>('nop');
sw = Stopwatch()..start();
for (var i = 0; i < n; i++) { noop(); }
print('ffi noop:     ${sw.elapsedMicroseconds * 1000 / n} ns/call');
```

What you will find, and what you should say.

A channel call is dominated by serialization plus a thread hop, so it lands in the microseconds-to-low-milliseconds range and grows with payload size and with how busy the two threads are.

An FFI call is a direct call into machine code already in your address space: no encoding, no hop, no queue. Orders of magnitude cheaper, and flat in payload size.

Quoted numbers disagree.

One slide of last year's deck said 0.5–2 ms; the next said 2 µs. Both unsourced, a factor of a thousand apart, and neither one described your phone.

Report the shape. Measure the number.

Calling C directly

```
// C: libsig.c, compiled and linked into the app
// typedef struct { double mean; double peak; } Stats;
// void analyze(const double *xs, int32_t n, Stats *out);
final class Stats extends Struct {          // the Dart view of it
  @Double() external double mean;
  @Double() external double peak;
}
final lib = Platform.isAndroid
    ? DynamicLibrary.open('libsig.so')      // Android: shared object
    : DynamicLibrary.process();             // iOS: statically linked in
final _analyze = lib.lookupFunction<
    Void Function(Pointer<Double>, Int32, Pointer<Stats>),  // C signature
    void Function(Pointer<Double>, int, Pointer<Stats>)     // Dart side
>('analyze');
(double, double) analyze(Float64List xs) {
  final arena = Arena();                    // frees everything at the end
  try {
    final buf = arena<Double>(xs.length);
    buf.asTypedList(xs.length).setAll(0, xs);   // one copy in
    final out = arena<Stats>();
    _analyze(buf, xs.length, out);             // <- the direct call
    return (out.ref.mean, out.ref.peak);
  } finally { arena.releaseAll(); }
}
```

You own the memory.

There is no GC on the other side. Allocate, free, or let an Arena do it, and use NativeFinalizer when a native handle must outlive a function.

The call blocks the isolate.

The Dart thread becomes the native thread. A 30 ms C function is 30 ms of dropped frames, so run it on a worker isolate.

Android: FFI reaches C, not Java.

Calling a Kotlin API means Dart → C → JNI, with reference bookkeeping on top; for a one-shot call a MethodChannel is often cheaper. Use jnigen if you need it often.

iOS: Apple's toolchain links C, C++ and Objective-C into the same binary,

so the symbols are already in the process and DynamicLibrary.process() finds them.

Don't hand-write bindings:

ffigen generates all of the above from the C header.

Big buffers: move the pointer, not the bytes

A 4K camera frame is 3840 × 2160 × 4 bytes, about 33 MB, thirty times a second. Send that through a platform channel and it is copied at least three times: native buffer → codec buffer → Dart heap. The memory bus saturates, the CPU caches are flushed, and the frames you are rendering suffer for it.

External typed data

A Uint8List that is a view onto native memory instead of a copy of it. Dart reads the bytes the C code wrote: zero copies, and a finalizer frees them.

Ownership transfer

Isolate.exit and TransferableTypedData hand a buffer to another isolate as an O(1) page remap, not a deep copy. The sender loses access, which is the guarantee that makes it safe.

Texture registry

The native side renders into a GPU texture and registers it; Flutter composites it straight into the scene. Camera preview and video never touch the Dart heap at all.

Data big enough to matter should not be serialized at all. Share it, or transfer it.

Channels for control, FFI for compute

	Platform channels	dart:ffi
Call style	asynchronous message passing	synchronous direct call
Cost	encode + thread hop, grows with payload	a function call; flat in payload
Data	copied and re-encoded on both sides	shared memory through a pointer
Threading	hops to the platform thread and back	runs on the calling isolate's thread
Reaches	any Kotlin / Swift API, plugins included	C ABI only: C, C++, Rust, Go c-archive
Fails as	a PlatformException you can catch	a segfault that takes the app with it

Channels for control.
Low-frequency intents where the boundary is crossed once per user action: open a document, read the battery, ask for a permission, start a scan. Convenience beats speed, and the whole native SDK is available.

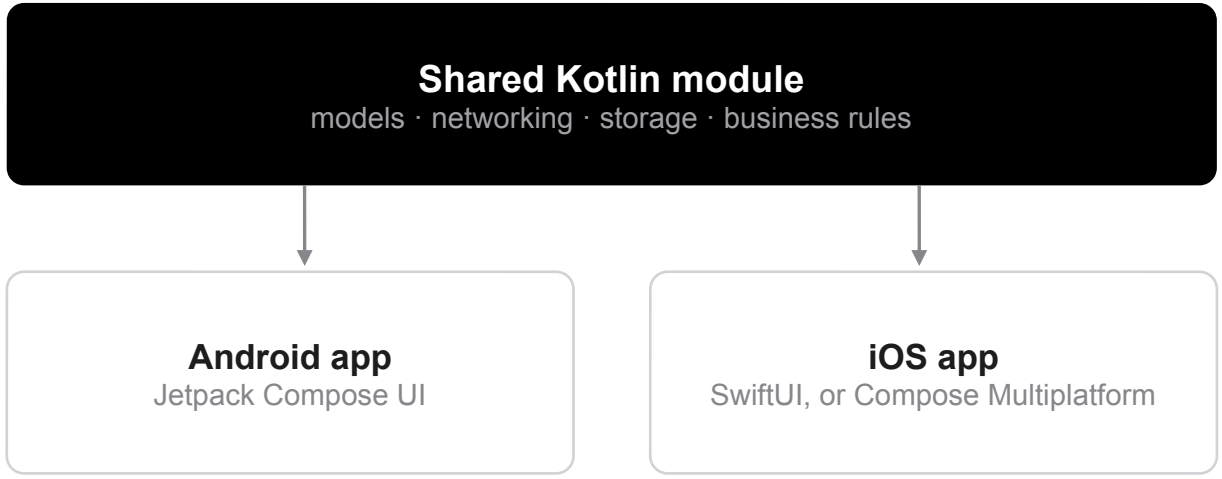
FFI for compute.
Anything on a per-frame or per-sample path: audio, signal processing, on-device inference, image pipelines. No encoding, no hop, and no GC on the other side, so the memory is yours to manage.

BOUNDARY FOUR

Between platforms

Kotlin Multiplatform: the shared module from Lecture 1

Share the logic, keep the UI



`expect / actual` covers the few pieces that must differ

Lecture 1 called this the third strategy. Here is what it actually looks like in a repository.

The Kotlin in that top box is not interpreted and not bridged. It is compiled by Kotlin/JVM for Android and by Kotlin/Native for iOS, into ordinary machine code the platform calls directly.

You choose how much goes in it. A models-and-networking module is a perfectly good result; so is everything but the views.

Stable since November 2023.

Compose Multiplatform for iOS stable since May 2025; Kotlin 2.4 is current.

In production at McDonald's, Netflix and Forbes

so it is a mature, low-risk choice.

expect and actual, in real code

commonMain: written once

```
// the shape, with no implementation
expect class KeyValueStore(name: String) {
    fun put(key: String, value: String)
    fun get(key: String): String?
}

// everything else is ordinary Kotlin, shared as-is
class ReadingsRepository(
    private val http: HttpClient,      // Ktor, multiplatform
    private val store: KeyValueStore,
) {
    suspend fun latest(deviceId: String): Reading {
        val cached = store.get(deviceId)
        return runCatching {
            http.get("$BASE/devices/$deviceId/latest").body<Reading>()
                .also { store.put(deviceId, Json.encodeToString(it)) }
        }.getOrElse { cached?.let(Json::decodeFromString) ?: throw it }
    }
}
```

iosMain calls NSUserDefaults directly: Kotlin/Native exposes the platform frameworks as Kotlin APIs, with no bridge and no wrapper library.

androidMain

```
actual class KeyValueStore actual constructor(name: String) {
    private val prefs = appContext
        .getSharedPreferences(name, Context.MODE_PRIVATE)
    actual fun put(key: String, value: String) =
        prefs.edit().putString(key, value).apply()
    actual fun get(key: String): String? =
        prefs.getString(key, null)
}
```

iosMain

```
actual class KeyValueStore actual constructor(name: String) {
    private val defaults = NSUserDefaults(suiteName = name)
    actual fun put(key: String, value: String) =
        defaults.setObject(value, forKey = key)
    actual fun get(key: String): String? =
        defaults.stringForKey(key)
}
```

Source sets: where the code lives

```
shared/
  build.gradle.kts
  src/
    commonMain/kotlin/     # most of the module lives here
    commonTest/kotlin/     # tests that run on every target
    androidMain/kotlin/
    iosMain/kotlin/        # iosArm64 + iosSimulatorArm64
androidApp/                # Compose UI
iosApp/                    # Xcode project, SwiftUI
```

A source set is just a folder the compiler includes for a given target. commonMain compiles for all of them, so anything platform-specific in it is a compile error, which is the point.

Dependencies follow the same rule: one Ktor API in common, a different engine per platform.

The build file is the architecture diagram.

```
// shared/build.gradle.kts
kotlin {
    androidTarget()
    listOf(iosArm64(), iosSimulatorArm64()).forEach { t ->
        t.binaries.framework {
            baseName = "Shared"
            isStatic = true    // one framework for Xcode
        }
    }
    sourceSets {
        commonMain.dependencies {
            implementation(libs.ktor.client.core)
            implementation(libs.kotlinx.coroutines.core)
        }
        androidMain.dependencies {
            implementation(libs.ktor.client.okhttp)
        }
        iosMain.dependencies {
            implementation(libs.ktor.client.darwin)
        }
    }
}
```

How the iOS app consumes it

```
# Kotlin/Native produces a real framework
./gradlew :shared:assembleSharedXCFramework

# then either:
#   - add the XCFramework to the Xcode project directly
#   - publish it as a Swift Package (SPM)
#   - or use the CocoaPods plugin's generated podspec
```

```
import Shared     // the Kotlin module, as an Obj-C / Swift API

struct DeviceView: View {
  @State private var reading: Reading?
  private let repo = ReadingsRepository(http: ..., store: ...)
  var body: some View {
    Text(reading.map { "\($0.tempC) °C" } ?? "n/a")
      // a Kotlin suspend fun arrives as Swift async/await
      .task { reading = try? await repo.latest(deviceId: id) }
  }
}
```

There is no bridge at run time.

Kotlin classes are compiled to native code and exposed as Objective-C classes, so Swift calls them the way it calls any other framework. Compare that with a platform channel, which encodes a message for every single call.

suspend maps to async/await;

Flow needs a small wrapper, or a tool like SKIE, to feel natural in Swift.

What you still pay:

someone on the team writes SwiftUI. Kotlin/Native's memory model is a tracing GC shared across threads: no manual freeing, but object graphs held from Swift keep Kotlin objects alive.

And the iOS build needs a Mac with Xcode,

exactly like a Flutter iOS build does.

Flutter and KMP, honestly compared

	Flutter	Kotlin Multiplatform
UI	one UI, drawn by Impeller on both platforms	each platform's native UI, or Compose on both
Shared	everything, including the pixels	whatever you put in commonMain
Boundary	crossed at run time: a channel or FFI call per OS API	crossed at compile time: shared code is native code
Language	Dart for the whole app	Kotlin shared, plus Swift for the iOS UI
Team	one team can ship both platforms	needs someone comfortable on each platform
Maturity	stable, very large plugin ecosystem	stable since Nov 2023; Compose iOS since May 2025

Neither one deletes the boundary: they move it. Flutter crosses it at run time, once per call. KMP crosses it at compile time, once per build. Lecture 1 chose Flutter for one team and one UI; this is the trade you accepted.

Where the semester ends up

Looking back

Boundaries were the recurring cost all semester: process, network, language, platform

gRPC: one .proto contract, four call shapes, one warm connection, with L5's Go backend on the far end

Channels for control, FFI for compute, and measure both yourself

KMP crosses at compile time; Flutter crosses at run time

Every performance number in a slide deck is a property of someone else's hardware

Before the exam

Run the full chain once: .proto → protoc → Go server → generated Dart client

Run the channel-vs-FFI benchmark on your own phone, in release mode

Move one model and one repository from your project into a shared Kotlin module

Re-read L5, L8 and L10 as one story about the wire

Bring your project questions to the last office hours

Read more

grpc.io/docs · protobuf.dev/programming-guides/encoding

connectrpc.com · RFC 9114 (HTTP/3) · RFC 9000 (QUIC)

docs.flutter.dev/platform-integration: channels, FFI, ffigen

dart.dev/interop/c-interop

kotlinlang.org/docs/multiplatform.html

That is the course. Project demos and the exam are what is left, and office hours stay open until both are done.